SOD

PRÁTICA 2.2

TCP/IP

P2.2. TCP/IP. CLIENTE SERVIDOR

PREÁMBULO

El objetivo de esta práctica es que los estudiantes se familiaricen con los conceptos de cliente y servidor.

ACTIVIDADES

Implementar una aplicación cliente-servidor mediante la utilización de sockets cuya funcionalidad será la transmisión de archivos desde el servidor al cliente.

El ejercicio constará de dos partes (procesos):

- El servidor. Proceso que estará esperando a que le lleguen conexiones desde el cliente y, una vez establecida la conexión, transferirá al cliente el contenido del archivo que se solicita. El contenido del archivo será el código fuente de una páginas cualquiera de internet o un html generado por vosotros.
- El cliente. Proceso que establecerá la conexión con el servidor, enviará una solicitud para obtener el de fichero, recibirá el contenido y lo mostrará por pantalla.
- La comunicación debe seguir el protocolo http.

PLUS (no obligatorio):

- Hacer que el servidor tenga más de un archivo: index.html, app.html y web.html y que devuelva al cliente el que este solicite y si no lo encuentra pues que devuelva un 404.

El cliente se ejecutará en la máquina local y se invocará con la orden:

cliente IP_Servidor (127.0.0.1 si ambos están en el mismo ordenador)

El servidor se lanzará con la orden Servidor y debe estar en todo momento escuchando por el puerto 9999, preparado para aceptar conexiones. Tras una petición de conexión por parte de un cliente, debe crear un hijo que será el encargado de realizar todas las operaciones necesarias para la transferencia del archivo.

La puntuación del ejercicio será la siguiente:

- Implementación correcta del cliente y su estructura. (2 puntos)
- Implementación correcta del servidor y su estructura. (2 puntos)
- Transferencia del archivo. (3 puntos)

LLAMADAS AL SISTEMA RELACIONADAS CON LAS COMUNICACIONES EN RED

Estructuras necesarias

<sys/socket.h>, struct sockaddr.
<netinet/in.h>, struct in_addr; struct sockaddr_in (incluye un campo in_addr).
<sys/types.h>

Llamadas al sistema principales socket

socket

Permite establecer el canal de comunicación, está definida en <sys/socket.h> y necesita <sys/types.h>.

conector = socket(AF_INET,SOCK_STREAM,0) //(Familia protocolos, tipo transporte, protocolo).

bind

Se emplea para comunicar a la red la dirección del canal. Utiliza como parámetro la estructura sockaddr_in definida en el archivo de cabecera <netinet/in.h>

bind(conector, addr, addrlen) //(descriptor del conector, dirección servidor, longitud dir)

listen

Disponibilidad para recibir peticiones de servicio y dimensionar la cola de espera.

listen(conector, tamaño_cola) // (descriptor del conector, cola peticiones pendientes)

accept

Bloquea al servidor en espera de peticiones de conexión por parte de los clientes.

accept(conector, addr, addrlen) //(descriptor del conector, dir. servidor, longitud dir)

connect

Es invocada por el proceso cliente para establecer una conexión, el servidor debe estar esperando.

connect(conector_cliente, addr, addrlen) // (descriptor conector, dir**ección servidor, longitud)**

Funciones de ayuda

inet_addr(cadena) //traduce una cadena de caracteres en dirección IP.
inet_ntoa(in_addr) //conversión inversa.
htonl, htons, ntohl, ntohs. Convierten datos unsigned long y unsigned short del formato de host al formato de la red y viceversa.

Protocolo http

HTTP (HyperText Transfer Protocol) es un protocolo de aplicación sin estado que define como un cliente (normalmente un navegador) y un servidor intercambian mensajes. Funciona sobre TCP (HTTP/1.1 y HTTP/2) o QUIC (HTTP/3).

- **HTTP/1.1**: texto plano, una petición por conexión.
- **HTTP/2**: binario, **multiplexación** de peticiones/respuestas en una sola conexión.
- **HTTP/3**: como HTTP/2 pero sobre **QUIC/UDP**, mejora latencia y resiliencia a pérdidas.
- **HTTPS** = HTTP + TLS (cifrado, autenticación del servidor, integridad).

Estructura de los mensajes (HTTP/1.1)

Petición (Request)

1. **Start-line**: <METHOD> <PATH or URL> HTTP/1.1
2. **Cabeceras**: Nombre-Cabecera: valor
3. **Línea en blanco**
4. **Cuerpo** (opcional, p.ej. en POST/PUT)

Respuesta (Response)

1. **Status-line**: HTTP/1.1 <STATUS-CODE> <REASON-PHRASE>
2. **Cabeceras**
3. **Línea en blanco**
4. **Cuerpo** (opcional, según código y recurso)

Métodos comunes

- **GET** (leer), **POST** (crear/enviar datos), **PUT** (reemplazar), **PATCH** (modificar parcial), **DELETE**, **HEAD**, **OPTIONS**.

Códigos de estado útiles

- **2xx** éxito: 200 OK, 201 Created, 204 No Content
- **3xx** redirección: 301 Moved Permanently, 302 Found, 304 Not Modified
- **4xx** error del cliente: 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
- **5xx** error del servidor: 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

Ejemplos HTTP_REQUEST (HTTP/1.1 en texto)

1) GET sin cabecera

```
GET /index.html HTTP/1.1
```

1) GET con cabecera

```
GET /index.html HTTP/1.1
Host: example.com
User-Agent: curl/8.7.1
Accept: application/json
Connection: keep-alive
```

Ejemplos HTTP_RESPONSE (HTTP/1.1 en texto)

GET con HTML

```
HTTP/1.1 200 OK
Server: Apache/2.0.40 (Red Hat Linux)
Content-Length: 256
Content-Type: text/html; charset=ISO-8859-1

<html>
<head>
<title>Tecnología Informática y Computación</title>
</head>
<body background="fondo.gif">
Hola a Todos<br>
<img src="menu.jpg">
</body>
</html>
```

POST con JSON

```
POST /api/v1/books HTTP/1.1

Host: example.com

Content-Type: application/json

Content-Length: 78


{

  "title": "A Wizard of Earthsea",

  "author": "Ursula K. Le Guin",

  "year": 1968

}
```

Cierre:

Para no tener que calcular el content-length, se el servidor puede añadir la cabecera Connection: close, escribir el cuerpo y cerrar el socket.
Esta es una plantilla valida de respuesta 200

```
HTTP/1.1 200 OK\r\n
Content-Type: text/html; charset=UTF-8\r\n
Connection: close\r\n
\r\n
<cuerpo...>
```

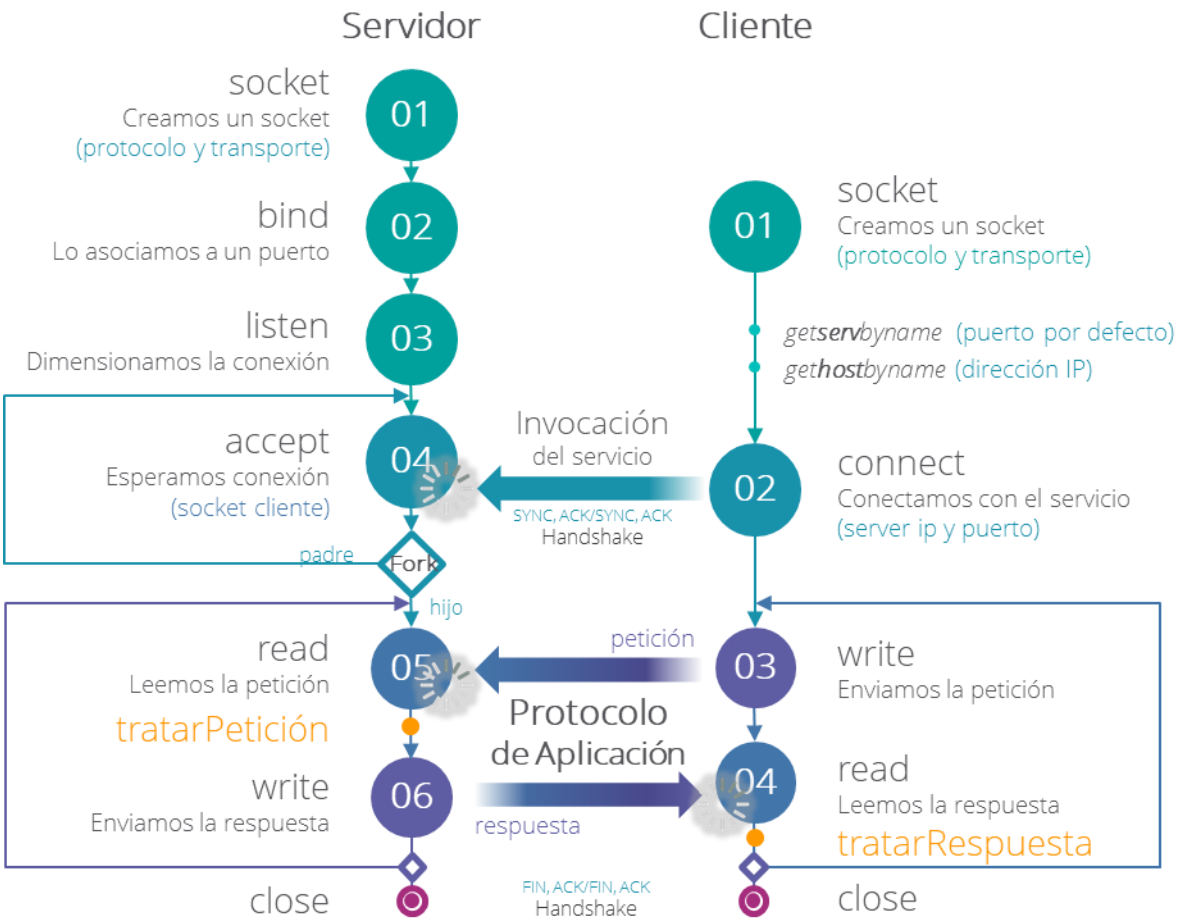
Plantilla de GET:

```
GET /index.html HTTP/1.1\r\n
```

HTML de ejemplo:

```
<!doctype html>
<html lang="es">
  <head>
    <meta charset="utf-8" />
    <title>SOD</title>
  </head>
  <body>
    <h1>Hola 👋 </h1>
    <p>Esto es una página HTML servida desde el servidor hecho en la practica 2</p>
    <p>La comunicación es HTTP por socket</p>
  </body>
</html>
```

Esquema cliente/servidor



EVALUACIÓN

- La entrega y presentación de esta segunda parte de la práctica será el 7 de noviembre.
- Hay que entregar el código de ambos programas (cliente y servidor) así como una pequeña memoria explicativa de las decisiones tomadas.
- La evaluación se realizará el día 7 de noviembre en la clase de prácticas.